

AIX

Large Language Models

Mingyu Kim,
May, 7th, 2026

Decoding-Only Transformers

Three Transformer families

- **Encoder-only models** build rich representations of the input and are strongest when the end task is classification, tagging, retrieval, or scoring.
- **Decoder-only models** predict the next token autoregressively, which makes them the natural choice for open-ended generation, dialogue, coding, and long-form completion.
- **Encoder-decoder models** are best when input and output play clearly different roles, such as translation or summarization, because the decoder can attend to a dedicated encoded source sequence.

Terminology

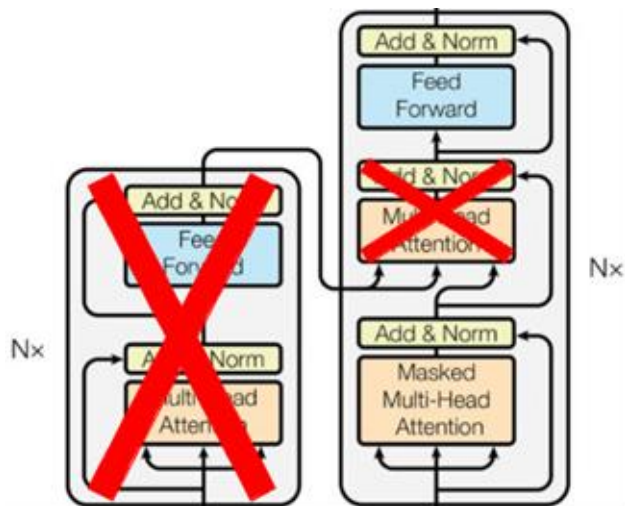
- LLM = Large Language Model
 - "A **language model** is a statistical or machine learning **model** that assigns **probabilities** to sequences of **tokens**."
 - "Large"
 - **Model size**: billions of parameters or more
 - Training data: 100s of billions of tokens or more
 - Compute: a lot of GPUs

What makes an LLM “large” in practice?

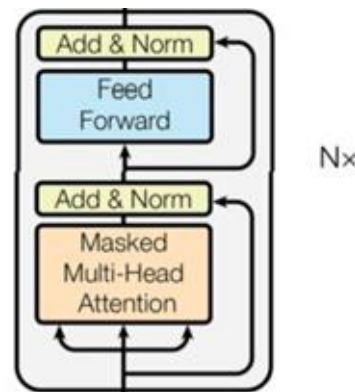
- Scale has three parts: parameter count, training data, and compute.
 - Strong performance usually comes from scaling all three together rather than maximizing only one of them.
- More parameters increase capacity, more data broadens coverage, and more compute lets the model actually absorb the patterns present in that data.
- The real reason “large” matters is not the number itself; it is the change in behavior: broader world knowledge, stronger few-shot behavior, and better zero-shot transfer across tasks.

Characteristics

- **Decoder-only Transformer-based model**
 - e.g) GPT series, LLaMA, Gemma, DeepSeek, Mistral, Qwen, ...



Encoder-Decoder Transformer



Decoder-Only Transformers

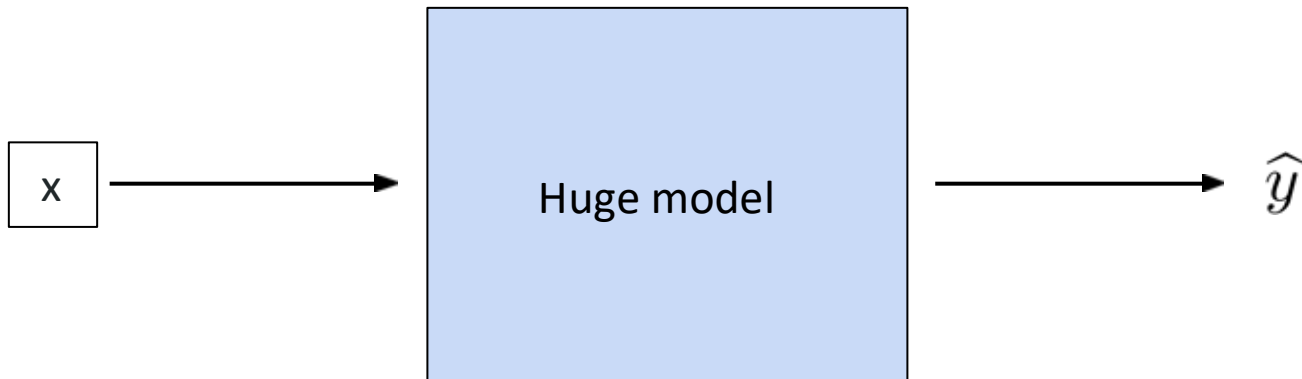
Why decoder-only models dominate open-ended generation

- Decoder-only models are trained with the same core objective used at inference: predict the next token from the tokens already seen. This tight train–serve alignment is one reason they scale well.
- A single autoregressive interface can handle chat, code completion, summarization prompts, tool calls, and creative writing without redesigning the architecture for each task.
- The weakness is sequential serving: decoding cannot be fully parallelized across future tokens, so latency grows token by token and becomes a systems problem as much as a modeling problem.

Mixture of Experts

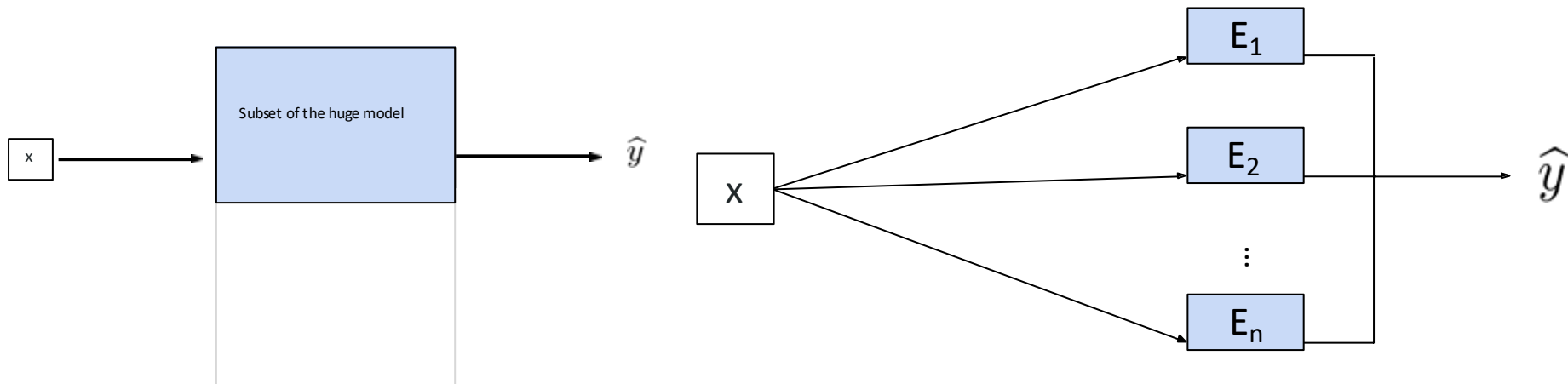
Mixture of Experts

- Why dense casing becomes inefficient?
 - In a dense model, every token activates every parameter block in the forward pass. Capacity and per-token compute therefore grow together.
 - This is wasteful because different tokens do not need the same kind of computation. A punctuation token, a code token, and a rare multilingual token may benefit from different specialists.



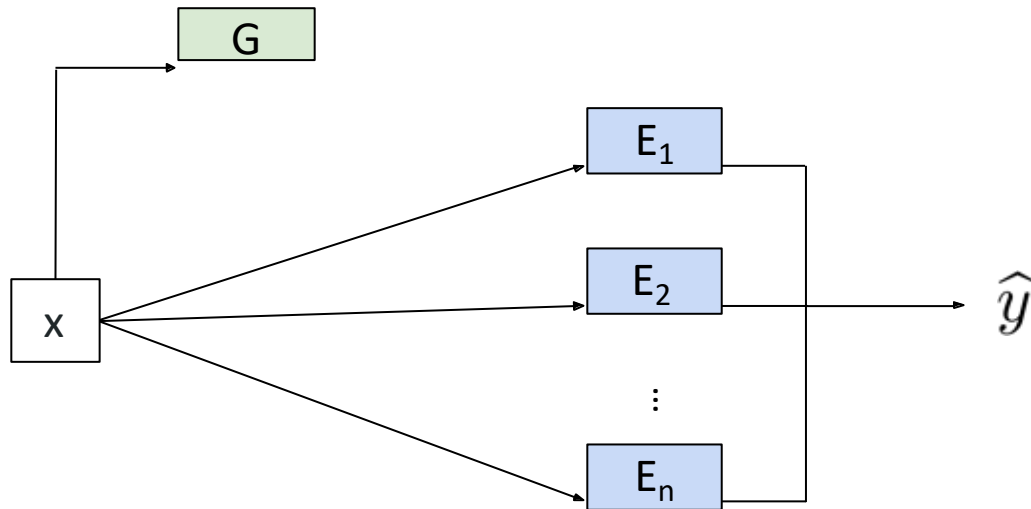
Mixture of Experts

- Idea : Not all weights are useful in the forward pass
 - Mixture-of-Experts separates total capacity from activated compute: the model can become much larger overall while only a small subset is used for each token.



Mixture of Experts

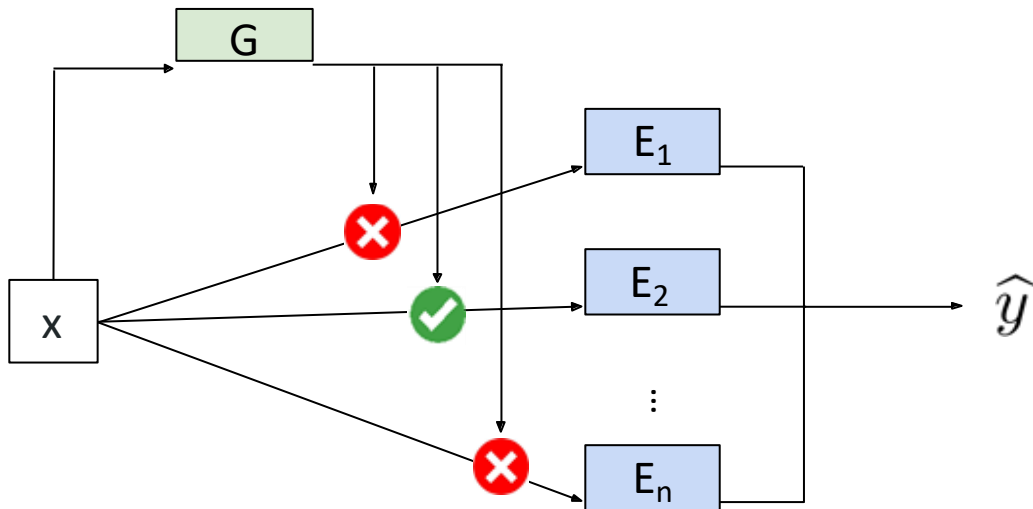
- A dense MoE combines outputs from all experts. It is easy to reason about, but it preserves most of the compute burden because every expert still runs.



Dense Mixture-of-Experts

Mixture of Experts

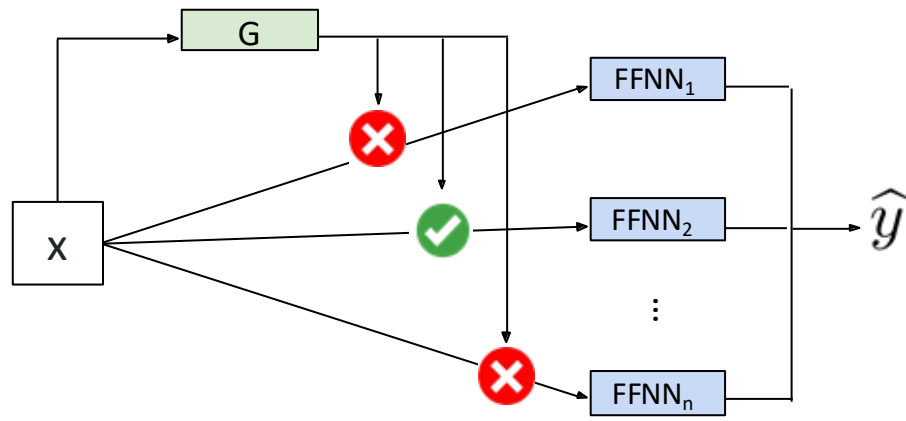
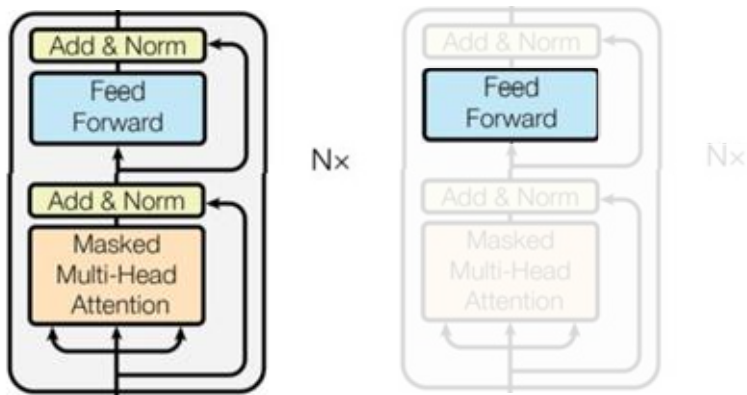
- A sparse MoE uses top-k routing so only the most relevant experts are activated for a token. This is the version that makes MoE attractive at scale.
- The trade-off is that routing quality now matters. Poor routing can erase the efficiency benefit or hurt accuracy even when the experts themselves are strong.



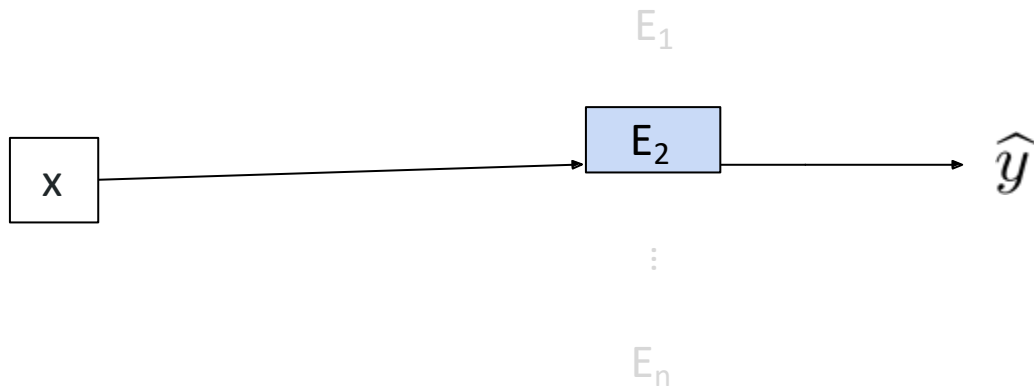
Sparse Mixture-of-Experts

Mixture of Experts

- Routing is done per token, not once per sentence. Tokens in the same sequence can therefore be sent to different experts depending on local needs.
 - Routing done **for each token!**
- Modern MoE LLMs usually replace the dense feed-forward sublayer rather than the attention sublayer, because the FFN dominates parameter count and is easier to specialize

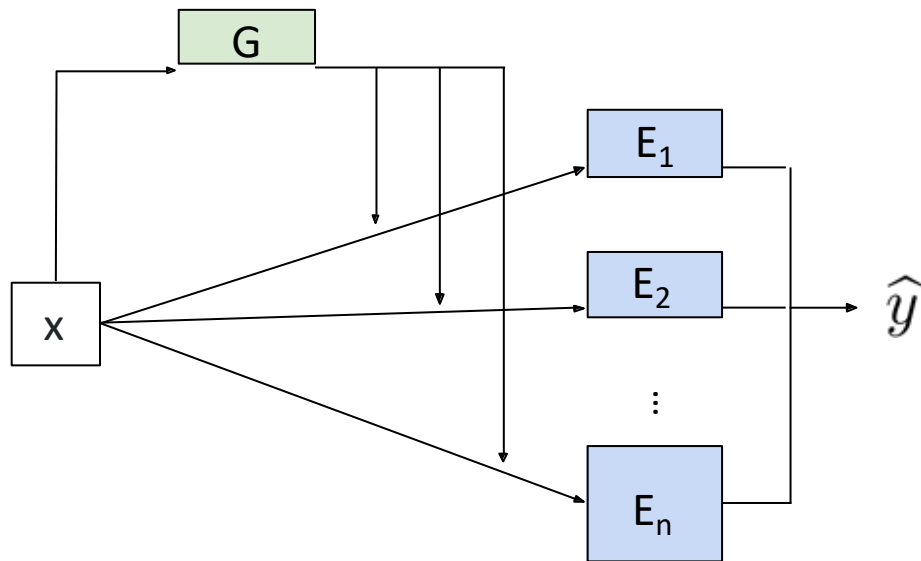


Mixture of Experts (Wrap Up)



Mixture of Experts (Wrap Up)

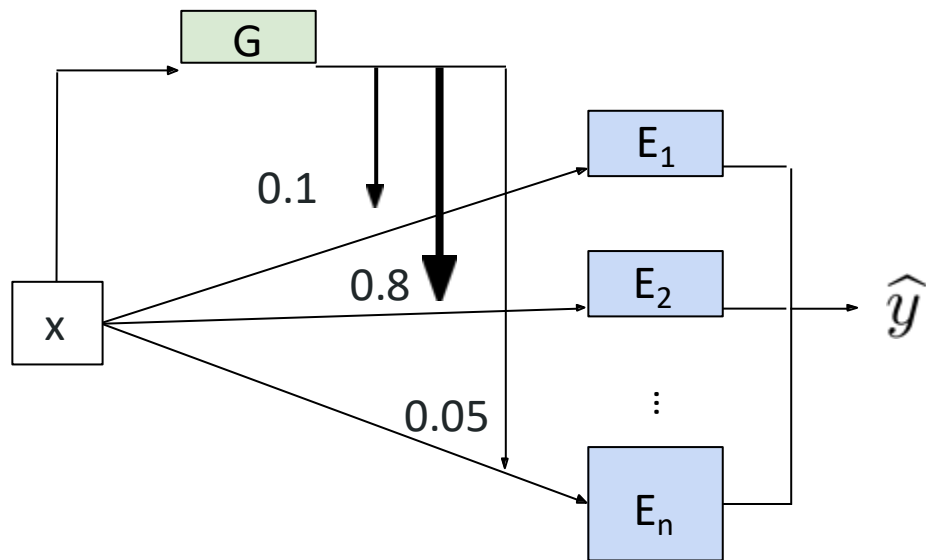
- Overview of MoEs



$$\hat{y} = \sum_{i=1}^n G(x)_i E_i(x)$$

Mixture of Experts (Wrap Up)

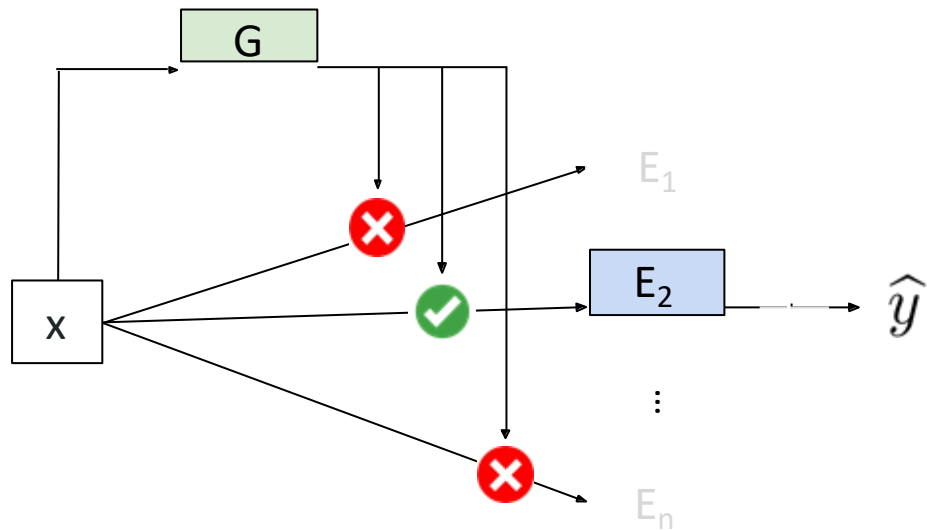
- Dense MoE. Output is weighted average of all expert outputs.



$$\hat{y} = \sum_{i=1}^n G(x)_i E_i(x)$$

Mixture of Experts

- Sparse MoE. Output is weighted average of selected expert outputs.
 - Top-k selection



$$\hat{y} = \sum_{i=1}^n G(x)_i E_i(x)$$

Mixture of Experts

- Training challenges : **"Routing Collapse"**
 - The router sends too many tokens to the same expert, leaving others underused or effectively dead.
 - This hurts both quality and efficiency. One expert becomes a throughput bottleneck while unused experts fail to learn meaningful specializations.

Mixture of Experts

- Remedy : Load Balancing
 - Auxiliary load-balancing losses encourage the model to spread tokens and routing probability more evenly, which keeps more experts active during training.
 - Force other experts to be "part of the game" via auxiliary loss

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i$$

f_i = fraction of tokens routed to expert

P_i = average routing probability for expert

Mixture of Experts

- Experts often specialize even without manual labels. Some activate more for code, some for multilingual text, some for punctuation-heavy regions, and some for frequent function words.
- This gives MoE an interpretability advantage: activation patterns can reveal how the model partitions the input space.
- At the same time, specialization is not perfectly clean. Experts are usually soft clusters rather than human-readable modules with a single neat semantic meaning.

Layer 0

```
class MoELayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
                 gate: nn.Module, moe_args):
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

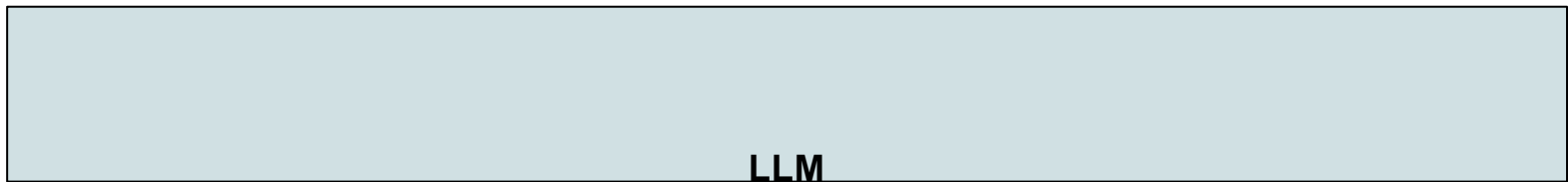
    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.size(-1))
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per_token
        )
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(
                selected_experts == i
            )
            results[batch_idx] += weights[batch_idx, nth_expert] * \
                expert(inputs_squashed[batch_idx])
        return results.view_as(inputs)
```

Next Token Prediction

From Logits to the Next Token

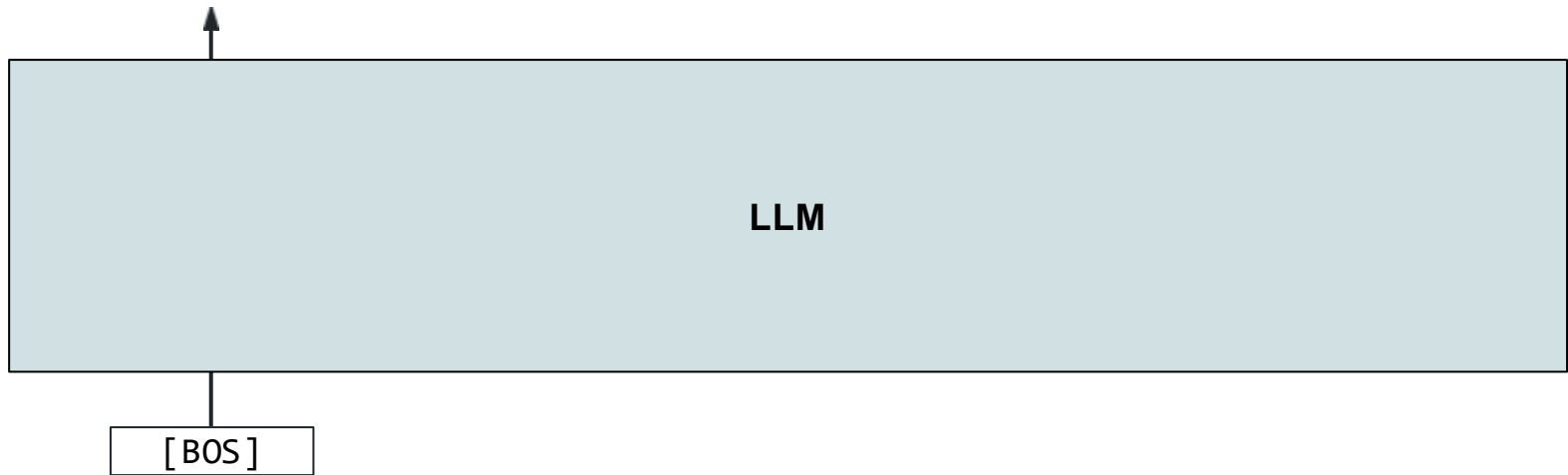
- A decoder-only LLM generates text one token at a time. At each step, the current context is mapped to logits over the vocabulary, then to a probability distribution.
- Training and inference are not identical: training parallelizes across many positions with teacher forcing, whereas inference must wait for the model's own previous outputs.
- Most decoding methods differ only in the final choice rule: do we choose the highest-probability token or sample from the distribution?

Next Token Prediction



[BOS]

Next Token Prediction



Next Token Prediction

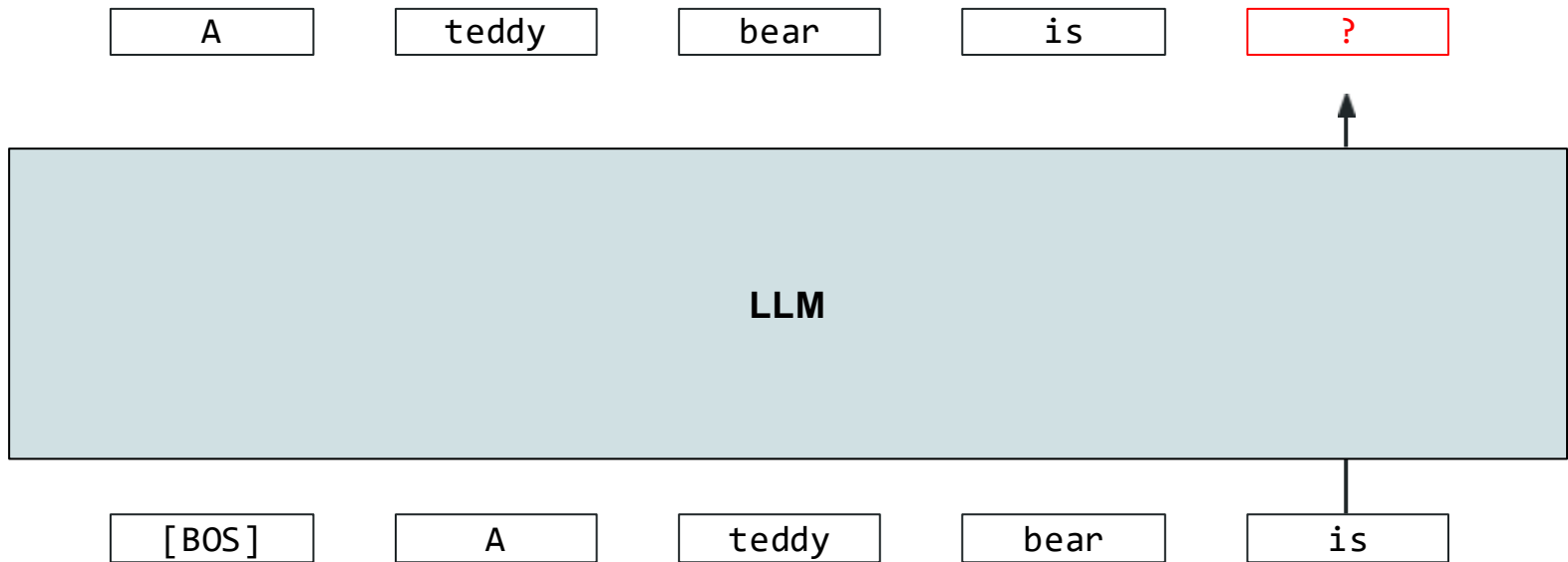
A

LLM

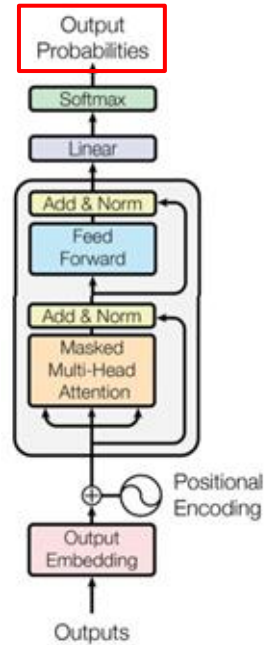
[BOS]

A

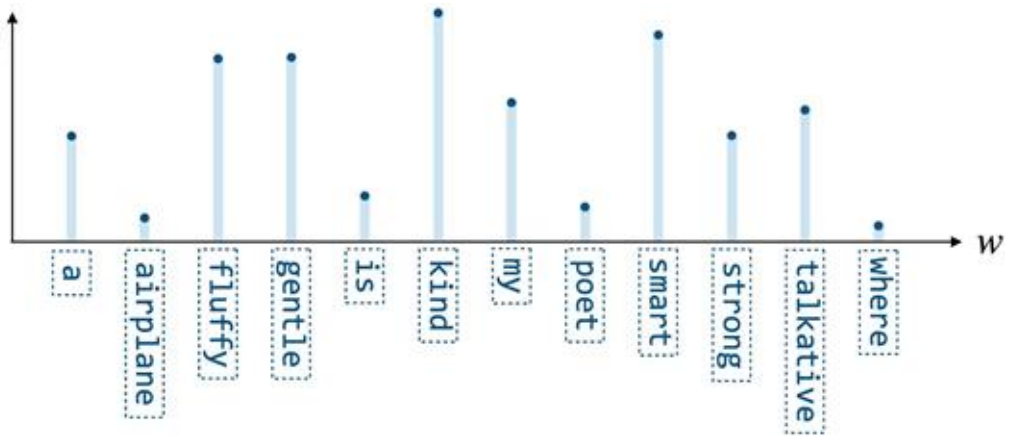
Next Token Prediction



Next Token Prediction : Greedy Decoding

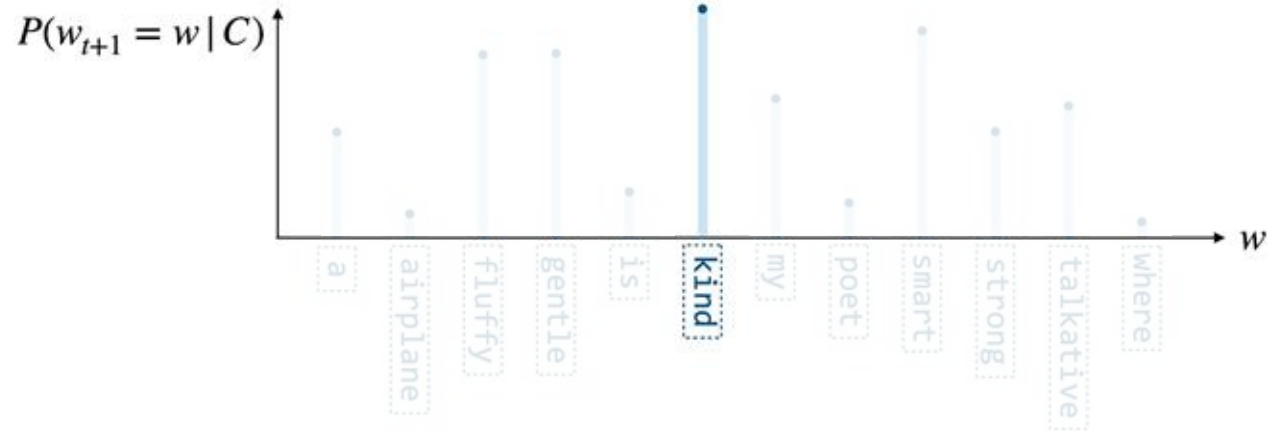
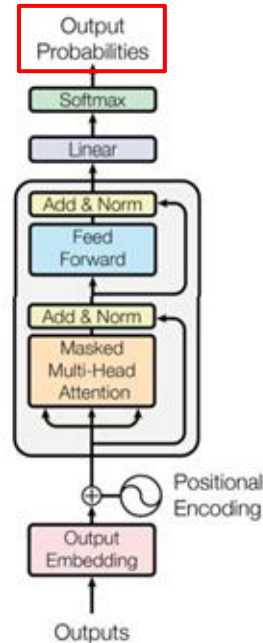


$$P(w_{t+1} = w | C)$$



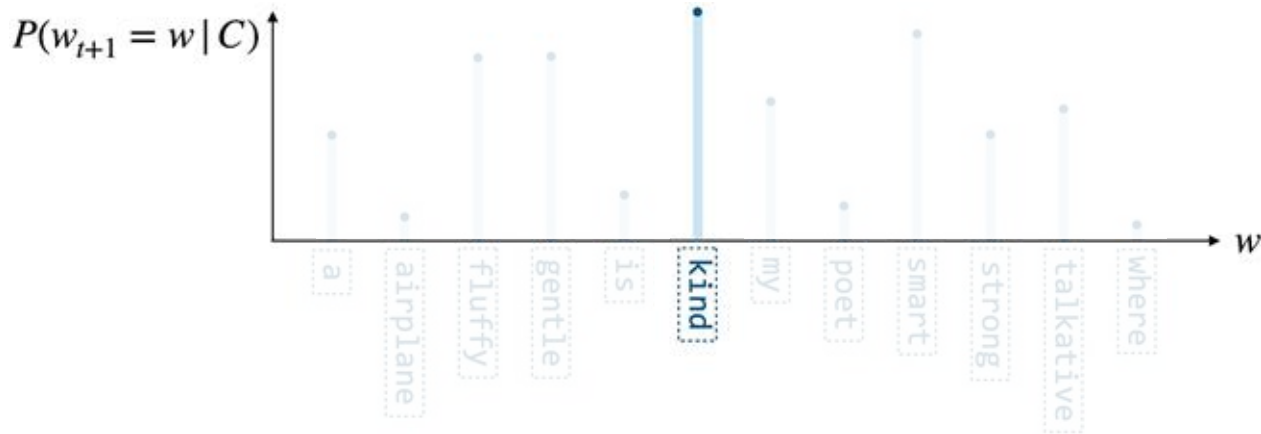
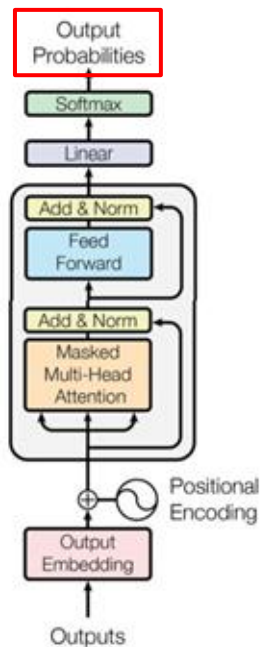
Next Token Prediction : Greedy Decoding

- 1st idea. Take token with highest predicted probability



Next Token Prediction : Greedy Decoding

- 1st idea. Take token with highest predicted probability
 - Limitations : Output not optimal, natural and/or diverse

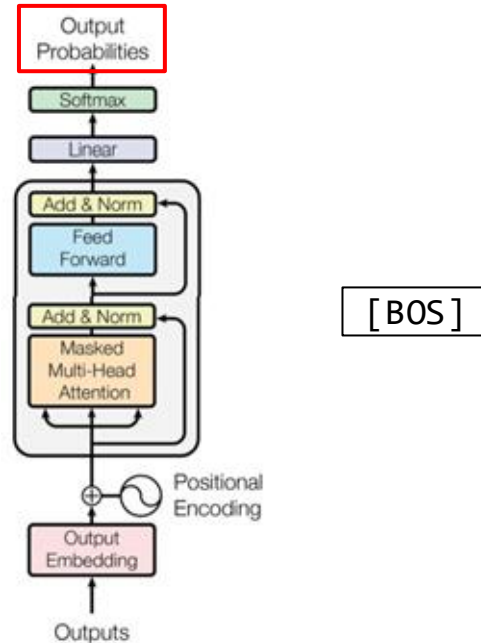


Next Token Prediction : Greedy Decoding

- Greedy decoding picks the single highest-probability token at every step, so it is deterministic, easy to implement, and very cheap to run.
- It works well when the task is narrow, the answer is highly constrained, or diversity is not valuable—for example, some extraction or rigid formatting tasks.
- Its weakness is myopia: a locally optimal token can push the model into a globally awkward, repetitive, or low-quality continuation.

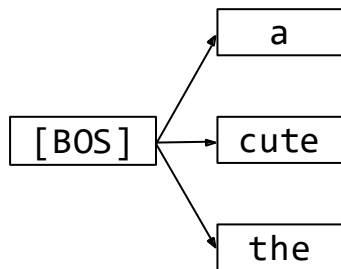
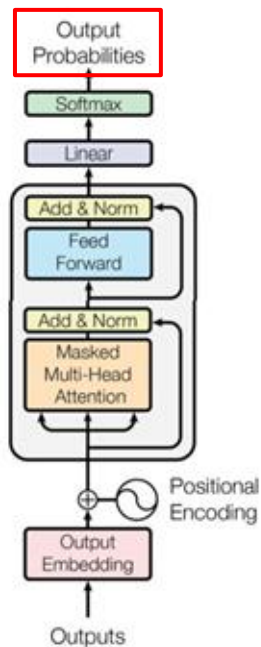
Next Token Prediction : Greedy Decoding

- 2nd idea : Keep k paths that are the most likely



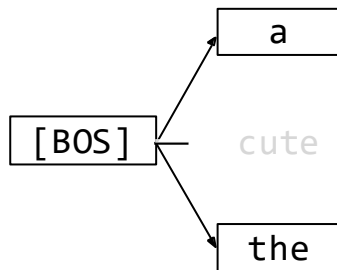
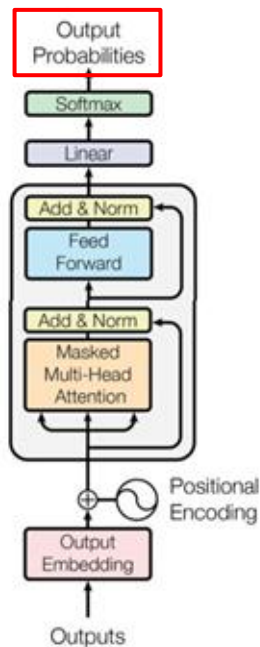
Next Token Prediction : Beam Search

- 2nd idea : Keep k paths that are the most likely



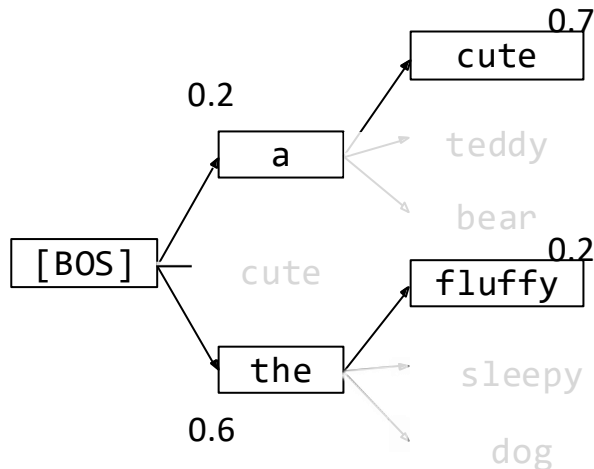
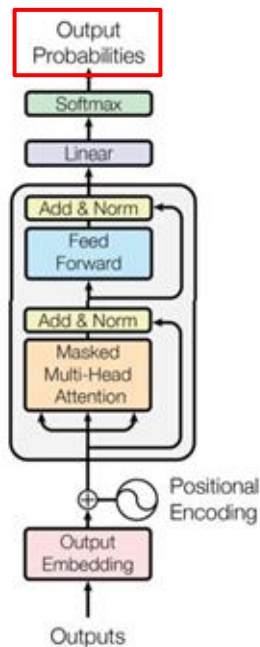
Next Token Prediction : Beam Search

- 2nd idea : Keep k paths that are the most likely



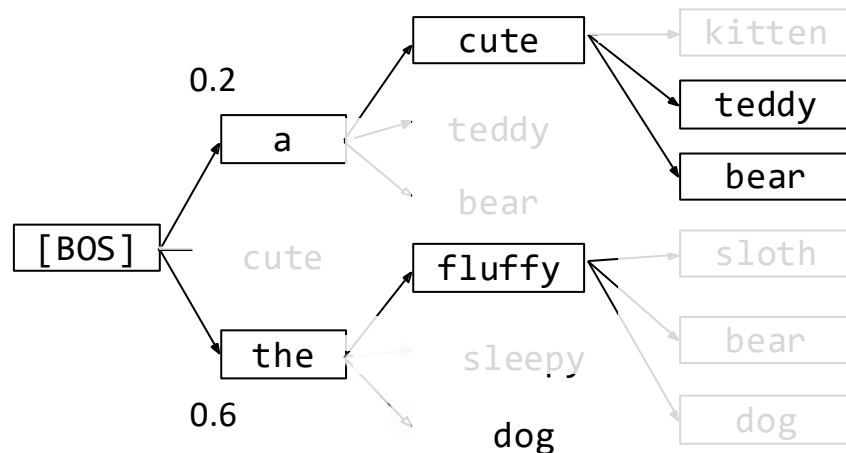
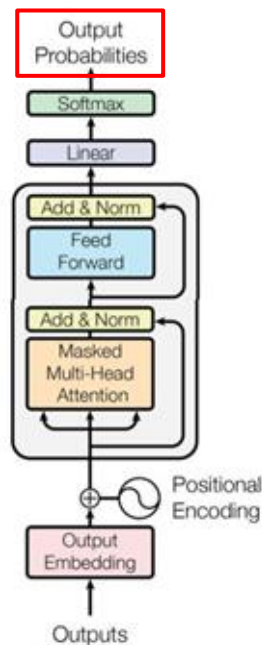
Next Token Prediction : Beam Search

- 2nd idea : Keep k paths that are the most likely



Next Token Prediction : Beam Search

- 2nd idea : Keep k paths that are the most likely
 - Limitations : Needs computations + lacks diversity/creativity



Ends at [EOS]

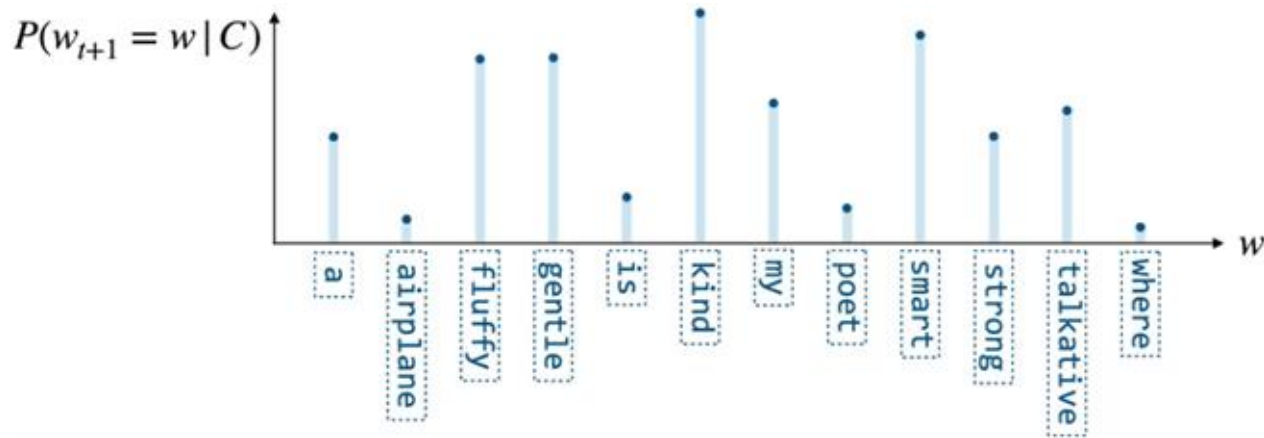
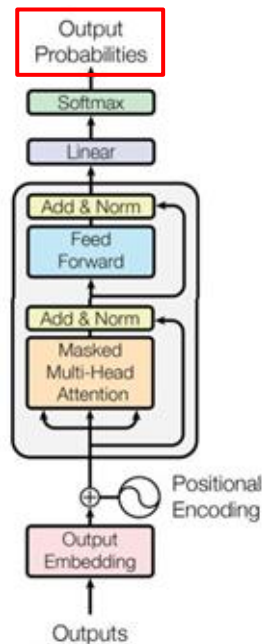
Next Token Prediction : Beam Search

- Beam search keeps the top k partial hypotheses instead of only one. This lets later evidence rescue a path that is not best after the very first decision.
- It is often more useful than greedy decoding for structured generation such as translation, where a better global sequence may require a non-greedy early step.
- The cost is higher latency and less diversity.
 - Many beams converge to similar continuations unless explicit diversity penalties or stochasticity are added.

Next Token Prediction : Sampling

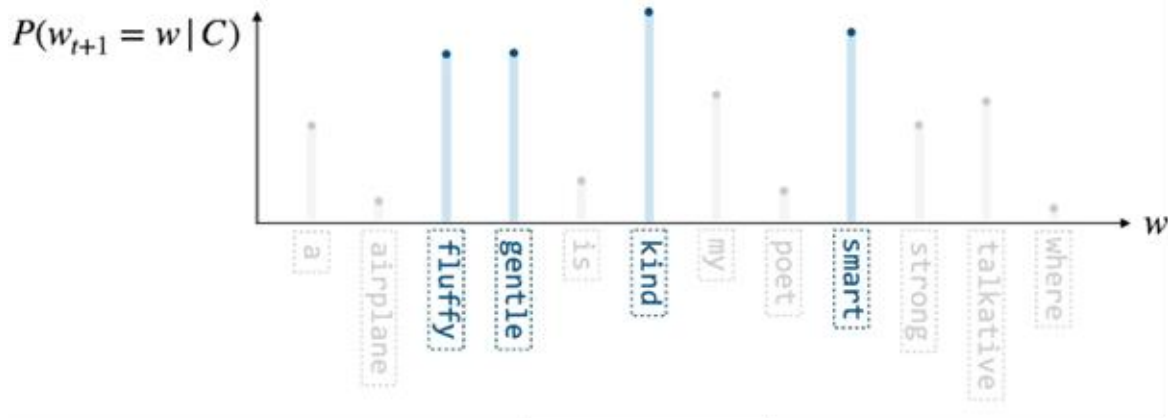
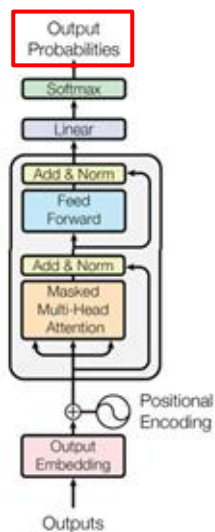
- 3rd idea. Sample next token from probability distribution:

$$\hat{w}_{t+1} \sim P(w_{t+1} | C)$$



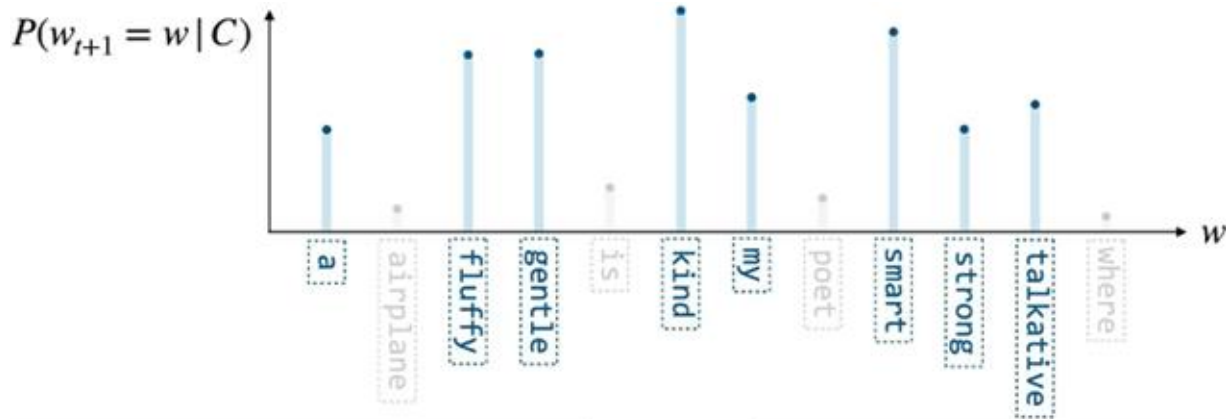
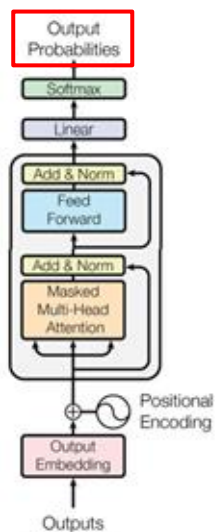
Next Token Prediction : Sampling

- 3rd idea. Sample next token from probability distribution:
 - Top-k: Sample among top k most probable tokens
 - e.g)



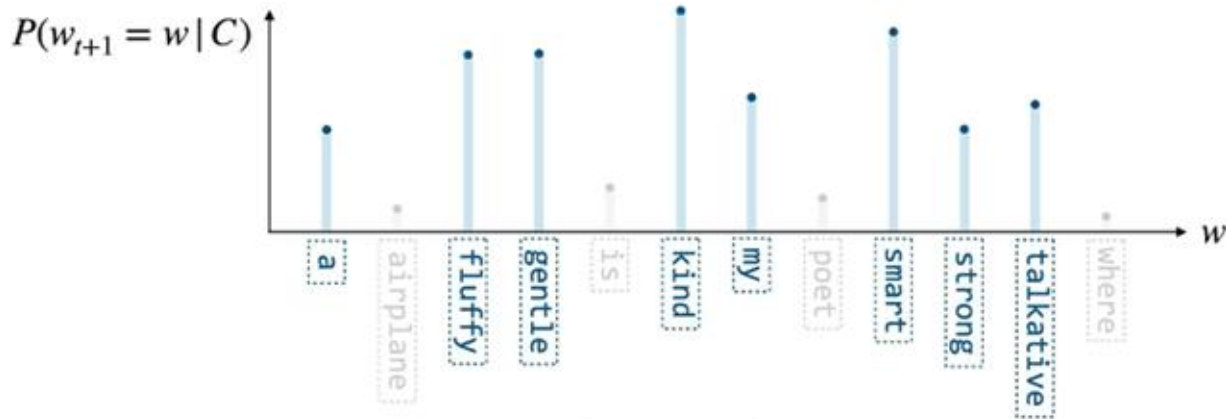
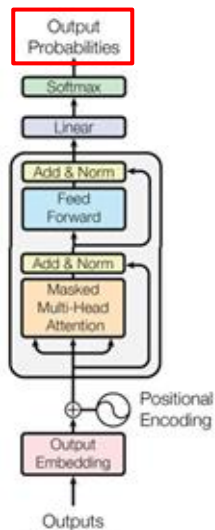
Next Token Prediction : Sampling

- 3rd idea. Sample next token from probability distribution:
 - Top-p: Random sample among smallest set of tokens with cumulative probability
 - e.g)



Next Token Prediction : Sampling

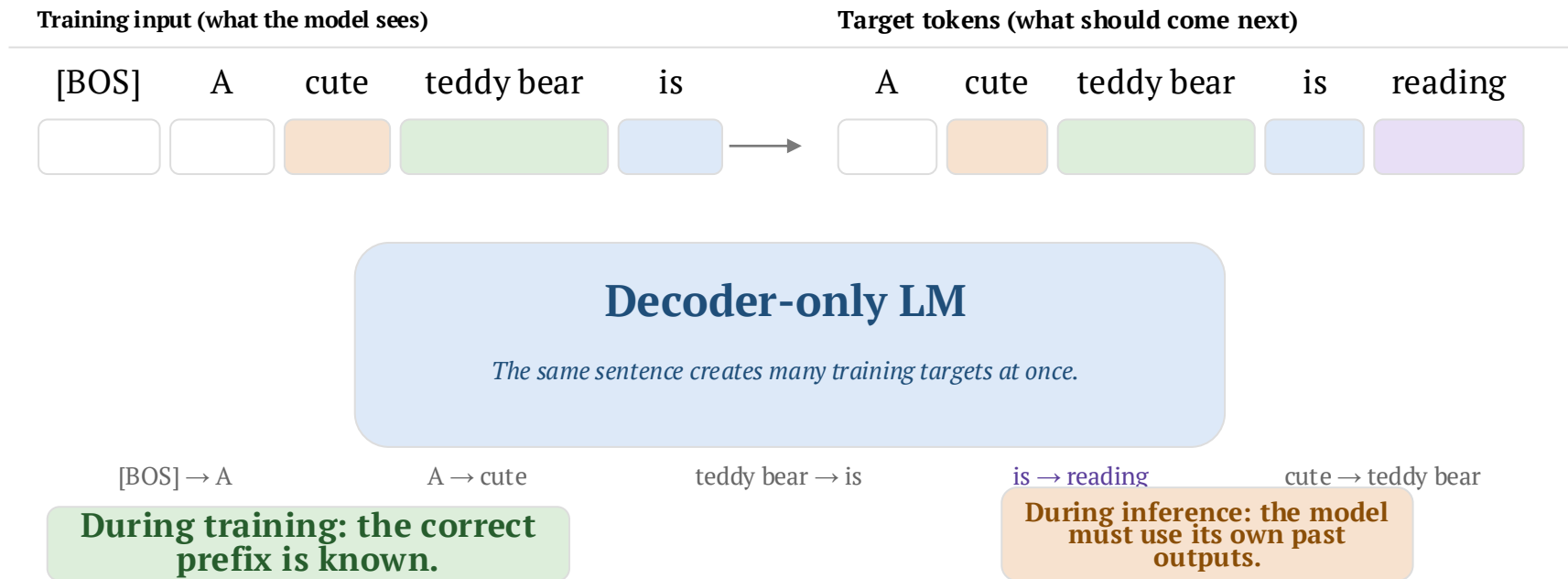
- 3rd idea. Sample next token from probability distribution:
 - Top-p: Random sample among smallest set of tokens with cumulative probability
 - e.g)



Next Token Prediction : Sampling

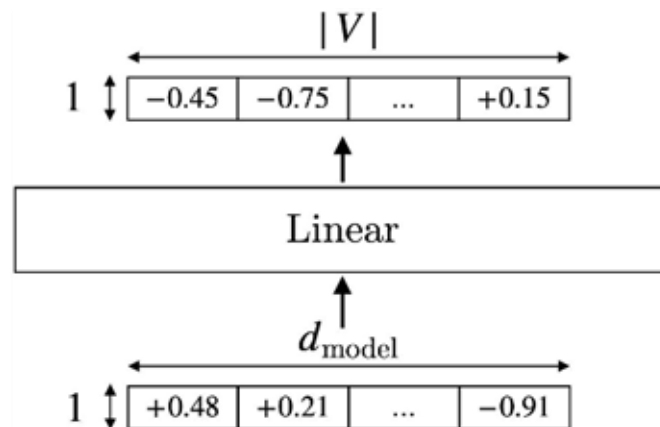
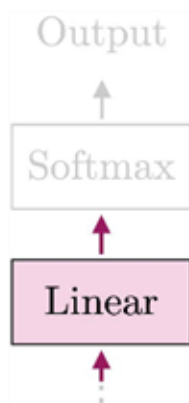
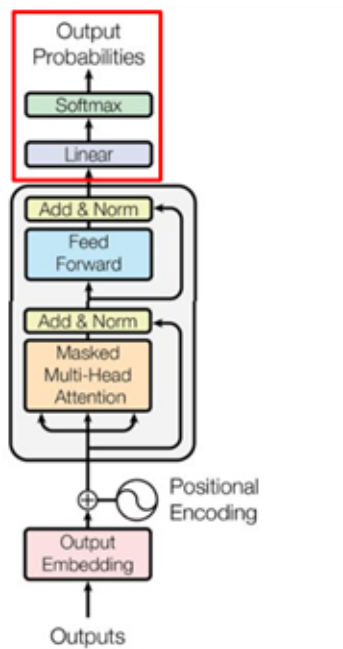
- For open-ended tasks, the most likely next token is not always the most useful or most human-like one.
 - Sampling lets the model explore other plausible continuations.
- Top-k keeps only the k most probable tokens, while top-p keeps the smallest set whose cumulative probability exceeds p . Both suppress the long tail of unlikely tokens.
 - Top-p is usually more adaptive because the candidate set size changes with uncertainty
 - Top-k is simpler but can be too narrow in some contexts and too broad in others.

Next-Token Prediction Uses Teacher Forcing During Pretraining

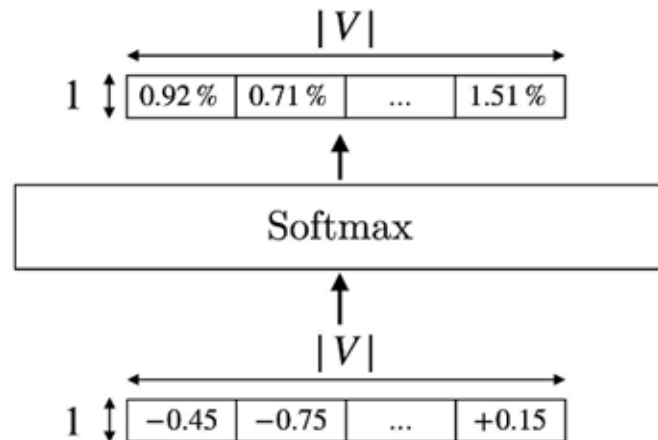
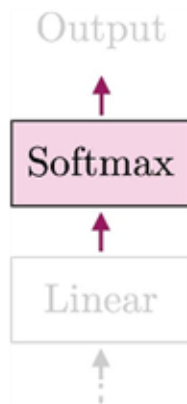
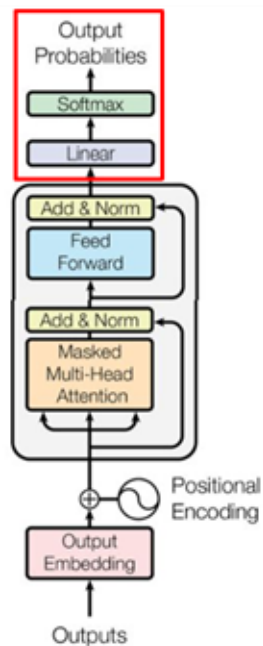


Training uses the true prefix; inference later uses the model's own generated prefix.

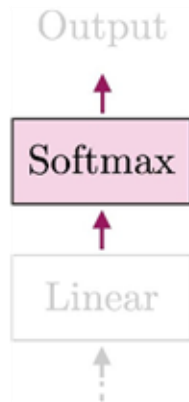
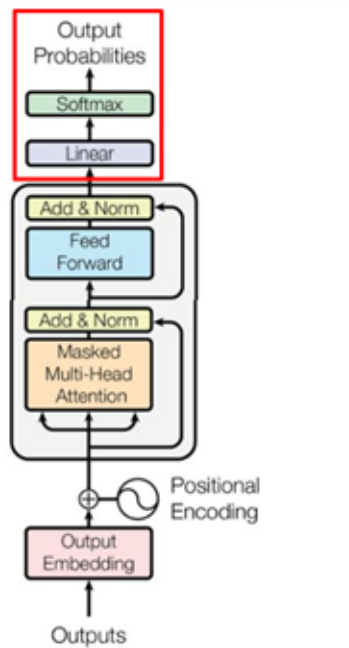
Temperature



Temperature



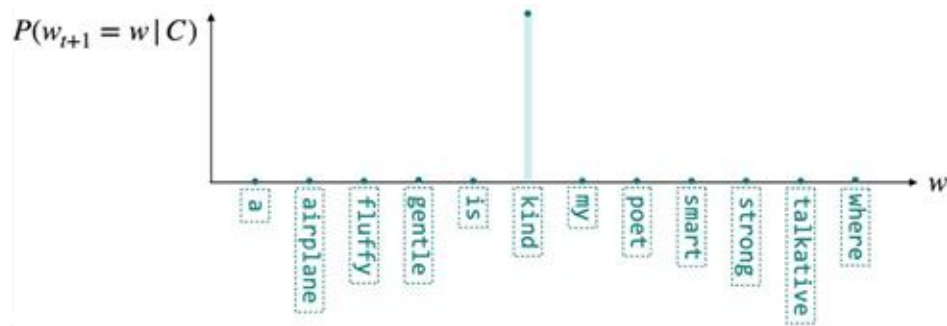
Temperature



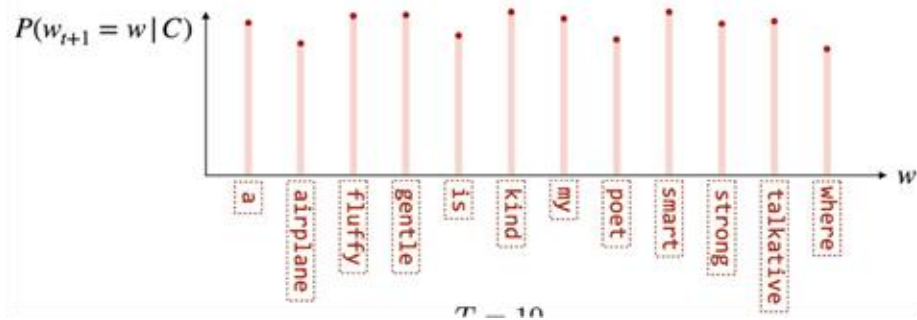
$$P_{\text{adj}}(w_{t+1} = w_i | C) = \frac{\exp\left(\frac{x_i}{T}\right)}{\sum_{j=1}^n \exp\left(\frac{x_j}{T}\right)}$$

Temperature

Small T



High T



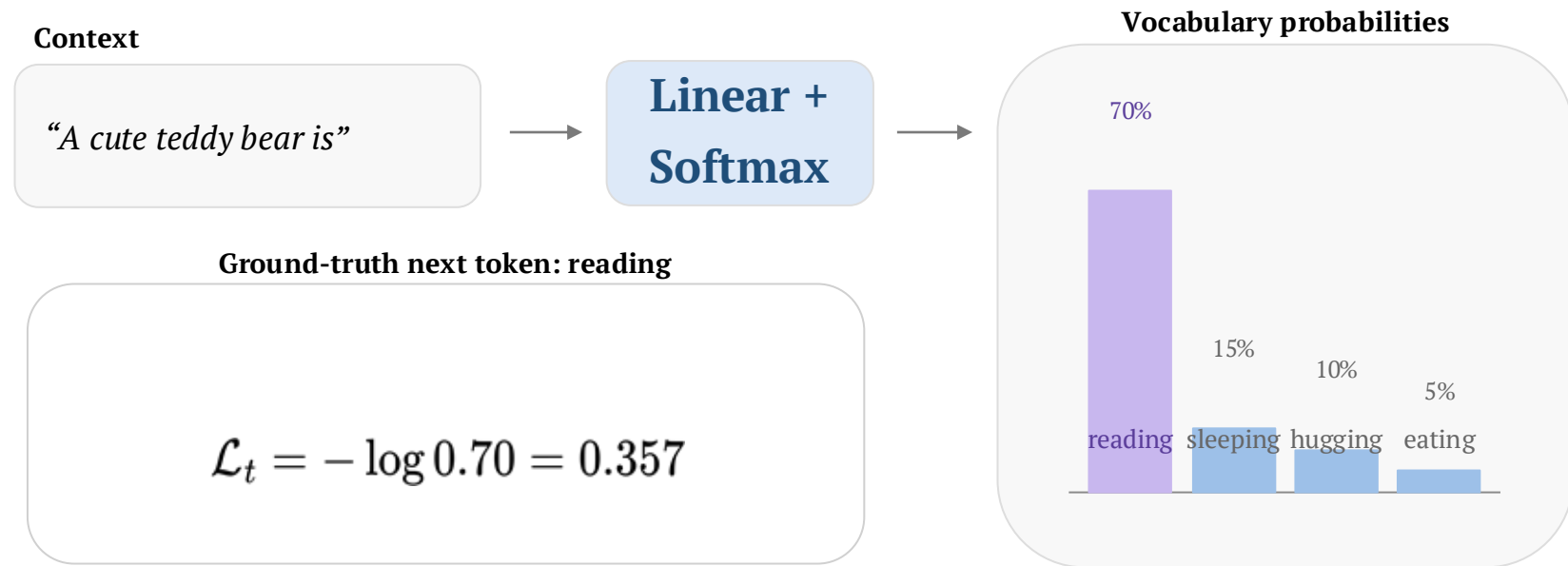
$T = 10$

Temperature changes entropy, not intelligence

- Temperature rescales logits before softmax. Lower values sharpen the distribution, while higher values flatten it and give low-probability tokens more chance to appear.
- Low temperature makes output more stable and repeatable, which is often useful for factual, evaluative, or tool-oriented tasks.
- High temperature increases stylistic range and creativity, but it also amplifies drift, inconsistency, and off-target responses when combined with a wide candidate set.

Cross-Entropy Loss for One Prediction Step

- At one position, the model outputs logits over the vocabulary and converts them into probabilities with softmax.



One prediction step already contains the core idea behind next-token pretraining.

Q&A

Any question?